UniSENAI

Trilha de Banco de Dados

Módulo 07: Segurança e Controle de Acesso

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.

2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.

3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.

4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.

5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.

6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.

7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.

8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 7: Segurança e Controle de Acesso

A segurança de dados é uma das prioridades em sistemas de banco de dados. O MySQL oferece ferramentas robustas para gerenciar usuários e suas permissões, garantindo que os dados sejam acessados apenas por quem possui autorização. Este módulo aborda como criar, gerenciar e proteger contas de usuários no MySQL, além de implementar boas práticas de segurança.

---

1. Gerenciamento de Usuários

O gerenciamento de usuários no MySQL é essencial para garantir que apenas pessoas ou sistemas autorizados acessem o banco de dados.

1.1 Criação de Usuários (CREATE USER)

O comando `CREATE USER` cria uma nova conta de usuário no MySQL, especificando o nome do usuário e a senha.

Sintaxe Geral

```
CREATE USER 'nome_usuario'@'host' IDENTIFIED BY 'senha';
```

- **`nome_usuario`**: Nome do usuário.
- **`host`**: Define de onde o usuário pode se conectar (`localhost` ou `%` para qualquer lugar).
- **`senha`**: Define a senha do usuário.

Exemplo Prático

Criar um usuário chamado `usuario1` com acesso local:

```
CREATE USER 'usuario1'@'localhost' IDENTIFIED BY 'senha_forte123';
```

Criar um usuário chamado `usuario_remoto` com acesso remoto:

```
CREATE USER 'usuario_remoto'@'%' IDENTIFIED BY 'senha_remota123';
```

Alteração e Exclusão de Usuários

1.2 Alteração de Usuários (ALTER USER)

O comando `ALTER USER` é usado para modificar atributos do usuário, como a senha.

Exemplo Prático: Alterar a senha de `usuario1`:

```
ALTER USER 'usuario1'@'localhost' IDENTIFIED BY 'nova_senha_forte456';
```

1.3 Exclusão de Usuários (DROP USER)

O comando `DROP USER` remove permanentemente uma conta de usuário do MySQL.

Exemplo Prático: Excluir o usuário `usuario_remoto`:

```
DROP USER 'usuario_remoto'@'%';
```

2. Concessão e Revogação de Permissões

Os comandos `GRANT` e `REVOKE` são usados para conceder ou revogar permissões aos usuários, definindo o que eles podem ou não fazer no banco de dados.

---

2. Concessão e Revogação de Permissões

As permissões determinam **o que um usuário pode ou não fazer** no banco de dados. O MySQL utiliza os comandos `GRANT` e `REVOKE` para gerenciar esses acessos.

2.1 Comando GRANT

Concede permissões específicas a um usuário.

Sintaxe Geral

```
GRANT tipo_de_permissao ON nome_do_banco.nome_da_tabela TO
'nome_usuario'@'host'
```

1. **`tipo_de_permissao`**: Permissões como `SELECT`, `INSERT`, `UPDATE`, `DELETE`, etc.
2. **`nome_do_banco.nome_da_tabela`**: Define o escopo das permissões.

Exemplo Prático

Conceder permissão de leitura (`SELECT`) na tabela `clientes` para `usuario1`:

```
GRANT SELECT ON meu_banco.clientes TO 'usuario1'@'localhost';
```

Conceder todas as permissões em um banco de dados:

```
GRANT ALL PRIVILEGES ON meu_banco.* TO 'usuario1'@'localhost';
```

2.2 Comando REVOKE

O comando `REVOKE` remove permissões previamente concedidas.

Sintaxe Geral

```
REVOKE tipo_de_permissao ON nome_do_banco.nome_da_tabela FROM 'nome_usuario'@'host';
```

Exemplo Prático

Revogar a permissão de leitura na tabela `clientes` de `usuario1`:

```
REVOKE SELECT ON meu_banco.clientes FROM 'usuario1'@'localhost';
```

Permissões de Acesso

Permissões comuns em MySQL incluem:

- **`SELECT`**: Permite consultar dados.
- **`INSERT`**: Permite inserir novos registros.
- **`UPDATE`**: Permite modificar registros existentes.
- **`DELETE`**: Permite remover registros.

---

3. Melhores Práticas de Segurança

Manter um banco de dados seguro envolve mais do que gerenciar usuários e permissões. É crucial adotar práticas de segurança robustas.

3.1 Uso de Senhas Fortes e Políticas de Senhas

- **Senhas Fortes**:
  - Devem conter pelo menos 8 caracteres.
  - Devem incluir letras maiúsculas, minúsculas, números e símbolos.
  - Exemplo: `Senha$F0rte!2024`

- **Configurar Políticas de Senhas**: Ative a política de senha forte no MySQL para garantir senhas seguras:

```
SET GLOBAL validate_password_policy = 'STRONG';
```

Configuração de Permissões de Usuário de Forma Segura

3.2 Princípio do Menor Privilégio:

- Conceda ao usuário apenas as permissões necessárias.
- **Exemplo**: Um usuário que só precisa ler dados deve receber apenas permissão `SELECT`.

```
GRANT SELECT ON meu_banco.* TO 'leitor'@'localhost';
```

3.3 Restrição de Origem (Host):

- Especifique o `host` de onde o usuário pode acessar.
- **Exemplo**: Permitir acesso apenas do servidor local:

```
CREATE USER 'usuario_local'@'localhost' IDENTIFIED BY 'senha123';
```

3.4 Evitar Usuários Compartilhados:

- Cada pessoa ou sistema deve ter sua própria conta de usuário.

3.5 Auditoria de Usuários e Permissões:

- Regularmente revise as contas e permissões para identificar excessos.

Aplicabilidade Prática

Sistemas de Gestão de Vendas:

- Um usuário pode ter acesso apenas às tabelas relacionadas aos pedidos, enquanto outro pode acessar apenas os clientes.

```
CREATE USER 'leitor_prod'@'%' IDENTIFIED BY 'senhaLeitor!';
GRANT SELECT ON producao.* TO 'leitor_prod'@'%';
```

Ambientes Multiusuário:

- Desenvolvedores podem receber permissões diferentes para realizar testes em tabelas específicas, enquanto administradores têm acesso total.

Ambientes de Produção:

- Use usuários separados para leitura e escrita para evitar alterações acidentais em dados críticos.

```
CREATE USER 'leitor_prod'@'%' IDENTIFIED BY 'senhaLeitor!';
GRANT SELECT ON producao.* TO 'leitor_prod'@'%';
```

Ao implementar estas práticas, é possível garantir que o banco de dados esteja protegido contra acessos não autorizados, mantendo a integridade e confidencialidade das informações.

Questões Práticas

- **Criação de Usuários**:
  Crie dois usuários no MySQL:
  - Um usuário chamado `usuario_local` com acesso apenas local (`localhost`).
  - Um usuário chamado `usuario_remoto` com acesso remoto (`%`).
- **Alteração de Usuários**:
  Modifique a senha do usuário `usuario_local` para uma senha forte, seguindo as práticas recomendadas.
- **Exclusão de Usuários**:
  Exclua o usuário `usuario_remoto` do banco de dados.
- **Concessão de Permissões**:
  - Conceda permissão de leitura (`SELECT`) na tabela `clientes` para o usuário `usuario_local`.
  - Conceda todas as permissões em um banco de dados chamado `loja` para o usuário `admin_user`.
- **Revogação de Permissões**:
  Revogue a permissão de leitura (`SELECT`) na tabela `clientes` do usuário `usuario_local`.
- **Configuração de Permissões por Host**:
  Crie um usuário chamado `usuario_servidor` com permissão de acessar o banco de dados apenas do servidor local.
- **Princípio do Menor Privilégio**:
  Configure um usuário chamado `relatorio_user` que tenha acesso somente à tabela `relatorios` com permissões de leitura (`SELECT`).
- **Auditoria de Usuários**:
  Liste todos os usuários criados no banco de dados e suas permissões utilizando os comandos adequados.

- **Políticas de Senhas**:
  Ative a política de senhas fortes no MySQL e aplique-a ao usuário `usuario_local`.
- **Configuração em Ambientes de Produção**:
  Configure dois usuários em um banco de dados de produção:
  - Um usuário com permissões apenas de leitura (`SELECT`).
  - Um usuário com permissões completas para inserção e alteração (`INSERT`, `UPDATE` e `DELETE`).

Questões Teóricas

- Explique a diferença entre os comandos `GRANT` e `REVOKE` no MySQL.
  - Dê exemplos de situações em que cada comando seria usado.
- O que é o princípio do menor privilégio? Por que ele é importante em sistemas de banco de dados?
- Qual é a importância de usar senhas fortes em bancos de dados? Cite as características de uma senha segura.
- Por que é recomendável evitar o uso de usuários compartilhados em bancos de dados?
- Quais são os riscos de conceder permissões desnecessárias a um usuário? Explique como evitar esses riscos.
- Explique como a restrição de acesso por host pode aumentar a segurança do banco de dados.
- Descreva as permissões comuns em MySQL (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) e dê exemplos de suas aplicações.
- O que é uma auditoria de usuários e permissões? Por que é importante realizá-la regularmente em um banco de dados?
- Quais práticas devem ser seguidas ao conceder permissões em um ambiente multiusuário?
- Considere um ambiente de produção onde há uma equipe de desenvolvedores e uma equipe de administradores. Como você configura os usuários e permissões para garantir segurança e eficiência?